

Nombre de la Universidad:
Anahuac Mayab Campus: Mérida

Actividad 2 - Carrito de compras

Nombre de los alumnos:
Santiago Loredó Cuevas
Emiliano Parra Varela
Eduardo Tellez Ramirez
Leonardo Argaez Ortega

Nombre del Docente:
Lizbeth Hernandez Olan

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?

Desarrollar un sistema de carrito de compras que funcione a través de una interfaz de consola, aplicando el paradigma de programación orientada a objetos (POO) y utilizando listas como estructura de datos principal para almacenar y gestionar la información en memoria temporal.

El programa debe simular la operación de una tienda permitiendo:

1. Consultar un catálogo precargado de productos y buscar artículos específicos por ID o nombre.
2. Agregar artículos al carrito especificando cantidades.
3. Visualizar el carrito en formato de tabla con cantidades, precios unitarios, subtotales por artículo y el total acumulado.
4. Modificar cantidades de productos previamente agregados, actualizando los subtotales automáticamente.
5. Eliminar productos de forma individual o vaciar por completo el carrito.
6. Finalizar la compra generando un resumen final que detalle los productos comprados, la cantidad de artículos y el monto total a pagar.

- ¿Qué se necesita?

Para construir e implementar la solución se requiere:

- **Entorno de desarrollo:** Un intérprete o compilador del lenguaje de programación seleccionado (por ejemplo, Python) con soporte para ejecución en consola.
- **Estructuras de datos en memoria:** Listas para almacenar el catálogo de productos disponibles y listas dinámicas para gestionar los artículos agregados al carrito de compras.
- **Paradigma Orientado a Objetos (POO):** Definición de clases que modelen las entidades del negocio (como los productos y el carrito de compras) encapsulando sus atributos y métodos.
- **Mecanismos de control y validación:**
 - Estructuras de control (bucles y condicionales) para la navegación del menú interactivo.
 - Validación de tipos de datos en las entradas del usuario (evitar bloqueos ante valores no numéricos o fuera de rango).
 - Confirmación de seguridad del usuario (s/n) para operaciones destructivas (eliminar artículo o vaciar el carrito).

- **Operaciones algorítmicas básicas sobre listas:** Inserción, búsqueda secuencial (por ID y nombre), actualización de valores y eliminación de elementos.

- **¿Qué elementos se identifican?**

Se identifican las siguientes entidades, estructuras de datos y componentes:

- **1. Entidad Producto (Catálogo):**
 - *Atributos:* **ID** (numérico entero), **Nombre** (texto), **Categoría** (texto) y **Precio** (decimal o flotante).
 - *Estructura:* Lista de productos precargada en memoria.
- **2. Entidad Elemento del Carrito / Carrito de Compras:**
 - *Atributos:* **Producto** (referencia o copia de los datos del producto), **Cantidad** (entero), **Subtotal** (calculado como: **precio * cantidad**).
 - *Totales acumulados:* Cantidad total de artículos y Monto general a pagar.
 - *Estructura:* Lista dinámica que almacena los productos agregados durante la sesión.
- **3. Componente de Interfaz / Interacción (Menú de consola):**
 - Navegación por 9 opciones principales:
 1. Mostrar catálogo
 2. Buscar producto
 3. Agregar producto al carrito
 4. Ver carrito
 5. Modificar cantidad
 6. Eliminar producto
 7. Vaciar carrito
 8. Finalizar compra
 9. Salir
- **4. Operaciones del Sistema:**
 - Búsqueda secuencial (filtrado por ID o coincidencia de texto).
 - Cálculo de subtotales y suma acumulada.
 - Eliminación por índice o búsqueda en la lista.
 - Limpieza total de la lista del carrito.

Fase 2. Definición de los requerimientos funcionales

Se debe elaborar el listado de los requerimientos del contexto planteado

Requerimientos
Nombre - R1 – Mostrar catálogo de productos
Descripción - El sistema debe mostrar en pantalla la lista completa de productos precargados en el catálogo, organizados en forma de tabla con sus datos principales.
Entrada - Selección de la opción 1 en el menú principal.
Salidas - Tabla con las columnas: ID, Producto, Categoría y Precio de cada artículo disponible en memoria.

Requerimientos
Nombre - R2 – Buscar producto en el catálogo
Descripción - El sistema debe permitir consultar la información de un producto específico mediante su ID numérico o mediante su nombre/descripción.
Entrada - Criterio de búsqueda (ID numérico o nombre/texto del producto).
Salidas - Datos del producto encontrado (ID, Nombre, Categoría, Precio) o un mensaje de error si el artículo no existe en el catálogo.

Requerimientos
Nombre - R3 – Agregar producto al carrito
Descripción - El sistema debe permitir seleccionar un producto disponible en el catálogo, especificar la cantidad deseada y agregarlo a la lista del carrito de compras. Si el producto ya se encontraba previamente en el carrito, se debe incrementar su cantidad y recalculan el subtotal.
Entrada - ID o nombre del producto a agregar, y un valor entero positivo correspondiente a la cantidad deseada.
Salidas - Confirmación en pantalla del producto seleccionado (Nombre y Precio

unitario), mensaje de éxito ("¡Producto agregado al carrito!") o aviso de producto inexistente / cantidad inválida.

Requerimientos

Nombre - R4 – Ver carrito de compras

Descripción - El sistema debe listar de forma estructurada y tabular todos los artículos contenidos en el carrito, detallando sus importes individuales y el costo acumulado de la compra.

Entrada - Selección de la opción 4 en el menú principal.

Salidas - Tabla con las columnas: ID, Producto, Precio, Cantidad y Subtotal por cada fila, seguida del importe general acumulado (TOTAL). Si el carrito no contiene artículos, muestra un mensaje indicando que está vacío.

Requerimientos

Nombre - R5 – Modificar cantidad de un producto en el carrito

Descripción - El sistema debe permitir localizar un producto agregado previamente al carrito, mostrar su cantidad registrada actualmente y actualizarla con un nuevo valor numérico, recalculando su subtotal en automático.

Entrada - ID (o nombre) del producto dentro del carrito y la nueva cantidad numérica (entero mayor a 0).

Salidas - Nombre del producto, cantidad actual y mensaje de confirmación ("¡Cantidad actualizada!").

Requerimientos

Nombre - R6 – Eliminar producto del carrito

Descripción - El sistema debe permitir dar de baja un artículo de la lista del carrito a partir de su ID o nombre, solicitando una confirmación de seguridad al usuario antes de proceder con el borrado.

Entrada - ID (o nombre) del producto a eliminar y confirmación del usuario con los caracteres **s** (sí) o **n** (no).

Salidas - Si se confirma con 's': mensaje de éxito ("¡Producto eliminado del carrito!"). Si se cancela con 'n': mensaje de operación cancelada. Si no existe:

mensaje de advertencia.

Requerimientos

Nombre - R7 – Vaciar carrito de compras

Descripción - El sistema debe eliminar la totalidad de los elementos guardados en la lista del carrito para reiniciar la sesión de compra, requiriendo previamente confirmación explícita del usuario.

Entrada - Confirmación interactiva mediante los caracteres **s** (sí) o **n** (no).

Salidas - Si se confirma con 's': mensaje de confirmación ("¡Carrito vaciado!") y limpieza de la lista en memoria. Si se responde 'n': mensaje de cancelación.

Requerimientos

Nombre - R8 – Finalizar compra

Descripción - El sistema debe procesar la compra actual generando el resumen de transacción final ("Ticket"), mostrando el desglose de productos adquiridos, el número de tipos de productos distintos, la suma total de unidades físicas de artículos y el importe definitivo a pagar.

Entrada - Selección de la opción 8 en el menú principal.

Salidas - Pantalla de "RESUMEN DE COMPRA" con la tabla de productos comprados (ID, Producto, Cantidad, Subtotal), la métrica de cantidad de productos distintos, la suma global de piezas/artículos, el Total general a pagar y el mensaje de agradecimiento.

Requerimientos

Nombre - R9 – Salir del sistema

Descripción - El sistema debe permitir terminar la sesión de consola de forma controlada, informando al usuario sobre la conclusión de la ejecución.

Entrada - Selección de la opción 9 en el menú principal.

Salidas - Mensaje de despedida ("Gracias por usar el programa. ¡Hasta pronto!")

Programa finalizado.") y cierre de la consola

Para la documentación de cada requerimiento funcional utilizar el siguiente esquema descriptivo.

Nombre	R1 - Nombre del requerimiento
Descripción	Especificar de forma detallada lo que hace el requerimiento
Entrada	La información que espera recibir
Salidas	El resultado esperado

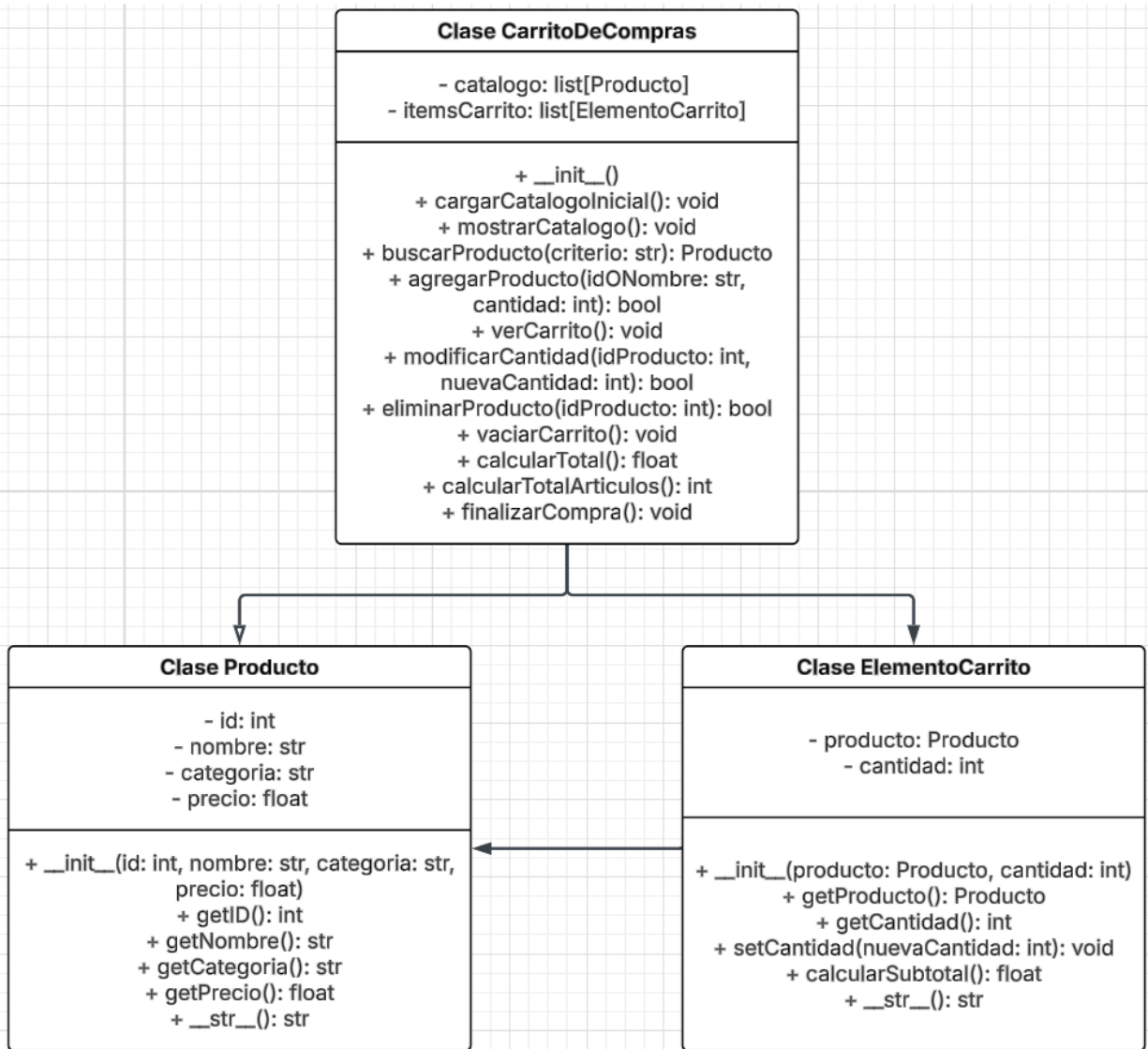
Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).

Nombre de la clase Producto
Atributos - id: int - nombre: str - categoria: str - precio: float
Métodos + __init__(id: int, nombre: str, categoria: str, precio: float) + getID(): int + getNombre(): str + getCategoria(): str + getPrecio(): float + __str__(): str

Nombre de la clase ElementoCarrito
Atributos - producto: Producto - cantidad: int
Métodos + __init__(producto: Producto, cantidad: int) + getProducto(): Producto + getCantidad(): int + setCantidad(nuevaCantidad: int): void + calcularSubtotal(): float + __str__(): str

Nombre de la clase CarritoDeCompras
Atributos - catalogo: list[Producto] - itemsCarrito: list[ElementoCarrito]
Métodos + __init__() + cargarCatalogoInicial(): void + mostrarCatalogo(): void + buscarProducto(criterio: str): Producto + agregarProducto(idONombre: str, cantidad: int): bool + verCarrito(): void + modificarCantidad(idProducto: int, nuevaCantidad: int): bool + eliminarProducto(idProducto: int): bool + vaciarCarrito(): void + calcularTotal(): float + calcularTotalArticulos(): int + finalizarCompra(): void



Fase 4. Codificación

Se deben escribir los programas fuentes necesarios, basados en el diagrama de clase de la fase anterior:

- Implementación de los métodos (construcción de la parte lógica).
- Codificación de fórmulas
- Asignaciones
- Condicionales
- Documentación del código

```
# Fase 4. Codificación - Sistema de Carrito de Compras
# Estructuras de Datos y Algoritmos - Universidad Anáhuac
import os
```

```
# CLASES DEL SISTEMA
```

```
class Producto:
```

```
    """Clase que modela los datos de un artículo del catálogo."""
```

```
    def __init__(self, id: int, nombre: str, categoria: str, precio: float):
```

```
        # Atributos definidos en el diseño UML
```

```
        self.id = id
```

```
        self.nombre = nombre
```

```
        self.categoria = categoria
```

```
        self.precio = float(precio)
```

```
    # Métodos Getter
```

```
    def getID(self):
```

```
        return self.id
```

```
    def getNombre(self):
```

```
        return self.nombre
```

```
    def getCategoria(self):
```

```
        return self.categoria
```

```
    def getPrecio(self):
```

```
        return self.precio
```

```
    def __str__(self):
```

```
        return f"{self.id} | {self.nombre} | {self.categoria} | ${self.precio:.2f}"
```

```
class ElementoCarrito:
    """Clase que representa un ítem dentro del carrito (asocia un producto con su cantidad)."""
    def __init__(self, producto: Producto, cantidad: int):
        self.producto = producto
        self.cantidad = int(cantidad)

    def getProducto(self):
        return self.producto

    def getCantidad(self):
        return self.cantidad

    def setCantidad(self, nuevaCantidad: int):
        self.cantidad = nuevaCantidad

    def calcularSubtotal(self):
        # Fórmula codificada para el subtotal
        return self.producto.getPrecio() * self.cantidad

    def __str__(self):
        return f"{self.producto.getID()} | {self.producto.getNombre()} |  

        ${self.producto.getPrecio():.2f} | {self.cantidad} | ${self.calcularSubtotal():.2f}"

class CarritoDeCompras:
    """Clase principal que administra el catálogo y las operaciones del carrito."""
    def __init__(self):
        # Uso obligatorio de listas como estructura de datos
        self.catalogo = []
        self.itemsCarrito = []
        self.cargarCatalogoInicial()

    def cargarCatalogoInicial(self):
        """Carga los productos predefinidos en la lista del catálogo en memoria."""
        self.catalogo.append(Producto(1, "Teclado", "Accesorios", 450.0))
        self.catalogo.append(Producto(2, "Mouse", "Accesorios", 250.0))
        self.catalogo.append(Producto(3, "Monitor", "Electrónica", 3200.0))
        self.catalogo.append(Producto(4, "Audifonos", "Audio", 800.0))
        self.catalogo.append(Producto(5, "Memoria USB", "Almacenamiento", 180.0))

    def mostrarCatalogo(self):
```

```
"""R1: Muestra en pantalla el catálogo completo."""
print("\n--- CATÁLOGO DE PRODUCTOS ---")
print("ID | Producto | Categoría | Precio")
print("-" * 40)
for prod in self.catalogo:
    print(prod)
print("-" * 40)

def buscarProducto(self, criterio: str):
    """R2: Busca un producto en el catálogo por ID o por nombre (búsqueda secuencial)."""
    criterioBusqueda = str(criterio).lower()
    for prod in self.catalogo:
        if str(prod.getID()) == criterioBusqueda or prod.getNombre().lower() ==
criterioBusqueda:
            return prod
    return None

def agregarProducto(self, idONombre: str, cantidad: int):
    """R3: Agrega un producto al carrito o incrementa la cantidad si ya existe."""
    if cantidad <= 0:
        print("ERROR. La cantidad debe ser mayor a 0.")
        return False

    producto = self.buscarProducto(idONombre)
    if not producto:
        print("ERROR. Producto no encontrado en el catálogo.")
        return False

    # Verifica si ya existe en el carrito
    for item in self.itemsCarrito:
        if item.getProducto().getID() == producto.getID():
            item.setCantidad(item.getCantidad() + cantidad)
            print(f"\n¡Producto actualizado! {producto.getNombre()} | Cantidad total:
{item.getCantidad()}")
            return True

    # Si no existe, lo inserta en la lista
    nuevotem = ElementoCarrito(producto, cantidad)
    self.itemsCarrito.append(nuevotem)
    print(f"\nProducto: {producto.getNombre()}")
    print(f"Precio unitario: ${producto.getPrecio():.2f}")
    print("¡Producto agregado al carrito!")
    return True
```

```
def verCarrito(self):
    """R4: Lista los artículos del carrito con sus subtotales y el total."""
    print("\n--- CARRITO DE COMPRAS ---")
    if not self.itemsCarrito:
        print("(El carrito está vacío)")
        return

    print(f'{'ID':<4} | {'Producto':<15} | {'Precio':<8} | {'Cant.':<5} | {'Subtotal':<10}}')
    print("-" * 55)

    for item in self.itemsCarrito:
        prod = item.getProducto()
        print(f'{'prod.getID():<4} | {'prod.getNombre():<15} | {'$prod.getPrecio():<7.2f} | {'$item.getCantidad():<5} | {'$item.calcularSubtotal():<9.2f}}')

    print("-" * 55)
    print(f'TOTAL: ${self.calcularTotal():.2f}')

def modificarCantidad(self, idProducto: int, nuevaCantidad: int):
    """R5: Actualiza la cantidad de un producto ya agregado."""
    if nuevaCantidad <= 0:
        print("ERROR. La cantidad debe ser mayor a 0.")
        return False

    for item in self.itemsCarrito:
        if item.getProducto().getID() == idProducto:
            print(f'Producto: {item.getProducto().getNombre()}')
            print(f'Cantidad actual: {item.getCantidad()}')
            item.setCantidad(nuevaCantidad)
            print("¡Cantidad actualizada!")
            return True

    print("ERROR. El producto no está en el carrito.")
    return False

def eliminarProducto(self, idProducto: int):
    """R6: Elimina un producto específico del carrito pidiendo confirmación."""
    for i in range(len(self.itemsCarrito)):
        if self.itemsCarrito[i].getProducto().getID() == idProducto:
            print(f'Producto: {self.itemsCarrito[i].getProducto().getNombre()}')
            confirmacion = input("¿Está seguro que desea eliminar este producto? (s/n): ")
            confirmacion = confirmacion.strip().lower()
```

```
        if confirmacion == 's':
            self.itemsCarrito.pop(i)
            print("¡Producto eliminado del carrito!")
            return True
        else:
            print("Operación cancelada.")
            return False

print("ERROR. El producto no está en el carrito.")
return False

def vaciarCarrito(self):
    """R7: Limpia por completo la lista del carrito con confirmación."""
    if not self.itemsCarrito:
        print("El carrito ya está vacío.")
        return

    self.verCarrito()
    confirmacion = input("\n¿Está seguro que desea vaciar el carrito? (s/n): ").strip().lower()
    if confirmacion == 's':
        self.itemsCarrito.clear()
        print("¡Carrito vaciado!")
    else:
        print("Operación cancelada.")

def calcularTotal(self):
    """Calcula la suma acumulada de todos los subtotales."""
    totalPagar = 0.0
    for item in self.itemsCarrito:
        totalPagar += item.calcularSubtotal()
    return totalPagar

def calcularTotalArticulos(self):
    """Calcula la suma física de unidades agregadas al carrito."""
    totalArticulos = 0
    for item in self.itemsCarrito:
        totalArticulos += item.getCantidad()
    return totalArticulos

def finalizarCompra(self):
    """R8: Procesa la compra y muestra el resumen de transacción final."""
    if not self.itemsCarrito:
        print("\nNo hay artículos en el carrito para comprar.")
```

```
        return

    print("\n--- RESUMEN DE COMPRA ---")
    print(f"{'ID':<4} | {'Producto':<15} | {'Cant.':<5} | {'Subtotal':<10}")
    print("-" * 45)
    for item in self.itemsCarrito:
        prod = item.getProducto()
        print(f"{'prod.getID():<4} | {'prod.getNombre():<15} | {'item.getCantidad():<5} | "
              f"${item.calcularSubtotal():<9.2f}")

    print("-" * 45)
    print(f"Productos distintos: {len(self.itemsCarrito)}")
    print(f"Total de artículos: {self.calcularTotalArticulos()}")
    print(f"Total a pagar: ${self.calcularTotal():.2f}")
    print("\nGracias por su compra.")

# --- FUNCIONES AUXILIARES Y CONTROL PRINCIPAL ---

def limpiarPantalla():
    """Limpia la consola según el sistema operativo."""
    os.system('cls' if os.name == 'nt' else 'clear')

def main():
    tienda = CarritoDeCompras()

    while True:
        limpiarPantalla()
        print("\n--- MENÚ PRINCIPAL - CARRITO DE COMPRAS ---")
        print("1. Mostrar catálogo")
        print("2. Buscar producto")
        print("3. Agregar producto al carrito")
        print("4. Ver carrito")
        print("5. Modificar cantidad")
        print("6. Eliminar producto")
        print("7. Vaciar carrito")
        print("8. Finalizar compra")
        print("9. Salir")

        opcion = input("\nSeleccione una opción: ").strip()

        if opcion == "1":
            limpiarPantalla()
```

```
tienda.mostrarCatalogo()
input("\nPresione ENTER para volver al menú...")

elif opcion == "2":
    limpiarPantalla()
    tienda.mostrarCatalogo()
    busqueda = input("\nBuscar producto (Ingrese ID o Nombre): ").strip()
    resultado = tienda.buscarProducto(busqueda)
    if resultado:
        print("\n--- Resultado de la búsqueda ---")
        print(f"ID: {resultado.getID()}")
        print(f"Producto: {resultado.getNombre()}")
        print(f"Categoría: {resultado.getCategoria()}")
        print(f"Precio: ${resultado.getPrecio():.2f}")
    else:
        print("\nERROR. Producto no encontrado.")
        input("\nPresione ENTER para continuar...")

elif opcion == "3":
    limpiarPantalla()
    tienda.mostrarCatalogo()
    idONombre = input("\nID o Nombre del producto a agregar: ").strip()
    try:
        cantidad = int(input("Cantidad: "))
        tienda.agregarProducto(idONombre, cantidad)
    except ValueError:
        # Manejo de errores básicos y validación
        print("ERROR. Debe ingresar un valor numérico entero para la cantidad.")
        input("\nPresione ENTER para continuar...")

elif opcion == "4":
    limpiarPantalla()
    tienda.verCarrito()
    input("\nPresione ENTER para volver al menú...")

elif opcion == "5":
    limpiarPantalla()
    tienda.verCarrito()
    try:
        idProducto = int(input("\nID del producto a modificar: "))
        nuevaCantidad = int(input("Nueva cantidad: "))
        tienda.modificarCantidad(idProducto, nuevaCantidad)
    except ValueError:
```

```
        print("ERROR. Los valores ingresados deben ser numéricos enteros.")
        input("\nPresione ENTER para continuar...")

elif opcion == "6":
    limpiarPantalla()
    tienda.verCarrito()
    try:
        idProducto = int(input("\nID del producto a eliminar: "))
        tienda.eliminarProducto(idProducto)
    except ValueError:
        print("ERROR. El ID debe ser un valor numérico entero.")
        input("\nPresione ENTER para continuar...")

elif opcion == "7":
    limpiarPantalla()
    tienda.vaciarCarrito()
    input("\nPresione ENTER para volver al menú...")

elif opcion == "8":
    limpiarPantalla()
    tienda.finalizarCompra()
    if tienda.itemsCarrito:
        tienda.itemsCarrito.clear()
    input("\nPresione ENTER para volver al menú...")

elif opcion == "9":
    limpiarPantalla()
    print("Gracias por usar el programa. ¡Hasta pronto!\nPrograma finalizado.")
    break

else:
    print("\nERROR. Opción no válida. Intente de nuevo.")
    input("Presione ENTER para continuar...")

if __name__ == "__main__":
    main()
```

Fase 5. Pruebas.

Aquí se debe:

- Verificar que el programa cumpla con los requerimientos establecidos.
- Construcción de los documentos de entrega final.
- Adjuntar evidencias de resultados de la ejecución (impresión de pantallas).

Menu inicial

```
input
--- MENÚ PRINCIPAL - CARRITO DE COMPRAS ---
1. Mostrar catálogo
2. Buscar producto
3. Agregar producto al carrito
4. Ver carrito
5. Modificar cantidad
6. Eliminar producto
7. Vaciar carrito
8. Finalizar compra
9. Salir
Seleccione una opción: 
```

Caso de Prueba 1: Opcion 1 - Mostrar catalogo

```
input
--- CATÁLOGO DE PRODUCTOS ---
ID | Producto | Categoría | Precio
-----
1 | Teclado | Accesorios | $450.00
2 | Mouse | Accesorios | $250.00
3 | Monitor | Electrónica | $3200.00
4 | Audifonos | Audio | $800.00
5 | Memoria USB | Almacenamiento | $180.00
-----
Presione ENTER para volver al menú...
```

Caso de Prueba 2: Opcion 2 - Buscar producto

```
input
--- CATÁLOGO DE PRODUCTOS ---
ID | Producto | Categoría | Precio
-----
1 | Teclado | Accesorios | $450.00
2 | Mouse | Accesorios | $250.00
3 | Monitor | Electrónica | $3200.00
4 | Audifonos | Audio | $800.00
5 | Memoria USB | Almacenamiento | $180.00
-----
Buscar producto (Ingrese ID o Nombre): 1
--- Resultado de la búsqueda ---
ID: 1
Producto: Teclado
Categoría: Accesorios
Precio: $450.00
Presione ENTER para continuar...
```

Caso de Prueba 3: Opcion 3 - Agregar producto al carrito

```
input
--- CATÁLOGO DE PRODUCTOS ---
ID | Producto | Categoría | Precio
-----
1 | Teclado | Accesorios | $450.00
2 | Mouse | Accesorios | $250.00
3 | Monitor | Electrónica | $3200.00
4 | Audifonos | Audio | $800.00
5 | Memoria USB | Almacenamiento | $180.00
-----

ID o Nombre del producto a agregar: 1
Cantidad: 1

Producto: Teclado
Precio unitario: $450.00
;Producto agregado al carrito!

Presione ENTER para continuar...
```

Caso de Prueba 4: Opcion 4 - Ver carrito

```
input
--- CARRITO DE COMPRAS ---
ID | Producto | Precio | Cant. | Subtotal
-----
1 | Teclado | $450.00 | 1 | $450.00
-----
TOTAL: $450.00

Presione ENTER para volver al menú...
```

Caso de Prueba 5: Opcion 5 - Modificar cantidad

```
input
--- CARRITO DE COMPRAS ---
ID | Producto | Precio | Cant. | Subtotal
-----
1 | Teclado | $450.00 | 1 | $450.00
-----
TOTAL: $450.00

ID del producto a modificar: 1
Nueva cantidad: 2
Producto: Teclado
Cantidad actual: 1
;Cantidad actualizada!

Presione ENTER para continuar...
```

Caso de Prueba 6: Opcion 6 - Eliminar producto

```
input
--- CARRITO DE COMPRAS ---
ID | Producto | Precio | Cant. | Subtotal
-----
1 | Teclado | $450.00 | 2 | $900.00
-----
TOTAL: $900.00

ID del producto a eliminar: 1
Producto: Teclado
;Está seguro que desea eliminar este producto? (s/n): s
;Producto eliminado del carrito!

Presione ENTER para continuar...
```

Caso de Prueba 7: Opcion 7 - Vaciar carrito

```
input
--- CARRITO DE COMPRAS ---
ID  | Producto      | Precio  | Cant. | Subtotal
-----
1   | Teclado       | $450.00 | 1     | $450.00
2   | Mouse        | $250.00 | 1     | $250.00
3   | Monitor       | $3200.00 | 1    | $3200.00
-----
TOTAL: $3900.00

¿Está seguro que desea vaciar el carrito? (s/n): s
¡Carrito vaciado!

Presione ENTER para volver al menú...
```

Caso de Prueba 8: Opcion 8 - Finalizar compra

```
input
--- RESUMEN DE COMPRA ---
ID  | Producto      | Cant. | Subtotal
-----
3   | Monitor       | 1     | $3200.00
4   | Audifonos     | 1     | $800.00
5   | Memoria USB   | 1     | $180.00
-----
Productos distintos: 3
Total de articulos: 3
Total a pagar: $4180.00

Gracias por su compra.

Presione ENTER para volver al menú...
```

Caso de Prueba 9: Opcion 9 - Salir

```
input
Gracias por usar el programa. ¡Hasta pronto!
Programa finalizado.

...Program finished with exit code 0
Press ENTER to exit console.
```